Java: História, Evolução e Mercado

Uma jornada completa pela linguagem que transformou o desenvolvimento de software — do passado ao presente, da teoria à prática.

PROGRAMAÇÃO ORIENTADA A OBJETOS

MERCADO DE TRABALHO

IDES & FERRAMENTAS

Origem e História do Java

1

1991 — O Início

James Gosling, Mike Sheridan e Patrick Naughton iniciam o **Projeto Green** na Sun Microsystems, com foco em dispositivos eletrônicos.

2

1995 — Lançamento Oficial

Java 1.0 é lançado ao público com a famosa promessa: **"Write Once, Run Anywhere"** — escreva uma vez, rode em qualquer lugar.

3

2010 — Oracle assume

A **Oracle Corporation** adquire a Sun Microsystems por aproximadamente $7,4 bilhões e passa a ser a responsável pelo desenvolvimento e manutenção do Java.

4

Impacto geral

Java democratizou o desenvolvimento multiplataforma e introduziu o gerenciamento automático de memória (**Garbage Collector**), o que reduziu drasticamente erros comuns de programação em linguagens como C++.

Principais Versões do Java

Cada versão trouxe inovações que moldaram a linguagem ao longo dos anos.

1 **Java 1.0 (1996)**
Versão inicial com a JVM e conceito multiplataforma.

2 **Java 5 (2004)**
Generics, anotações, enums e autoboxing — grande salto na expressividade.

3 **Java 8 (2014)**
Lambdas, Streams e a nova API de datas — versão mais popular da história.

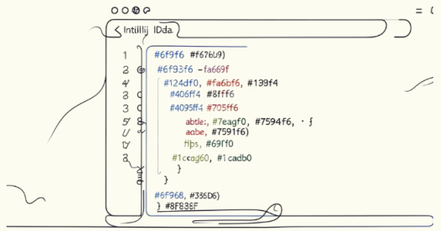
4 **Java 11 (2018)**
LTS com módulos, HTTP Client e melhorias de desempenho significativas.

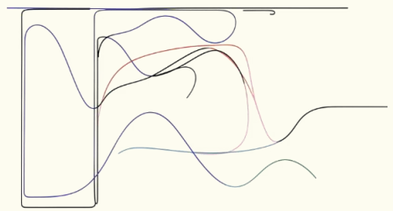
5 **Java 17 (2021)**
LTS com Records, Sealed Classes e melhorias de desempenho significativas.

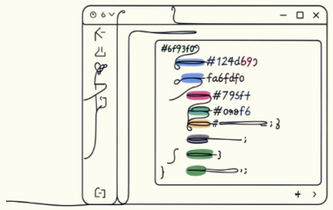
6 **Java 21 (2023)**
Virtual Threads, Pattern Matching e recursos modernos de concorrência.

Ambientes de Desenvolvimento (IDEs)

Escolher a IDE certa faz toda a diferença na produtividade do desenvolvedor Java.







IntelliJ IDEA

Site: https://www.jetbrains.com/idea/

✅ Considerada a mais inteligente. Possui autocompletar avançado, refatoração segura e excelente integração com frameworks.
❌ É pesada (consome muita RAM) e a versão completa (*Ultimate*) é paga.

Eclipse IDE

Site: https://www.eclipse.org/

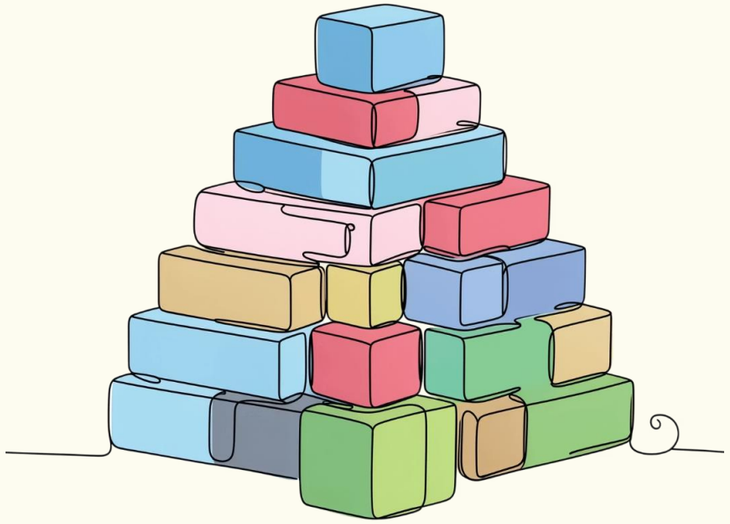
✅ Gratuita e possui uma comunidade gigante com plugins para quase tudo. É o padrão em muitas empresas.
❌ Interface mais pesada e desatualizada.

Apache NetBeans

Site: https://netbeans.apache.org/

✅ É a IDE "oficial" da Oracle por muito tempo. Ótima para aplicações Desktop (Swing/JavaFX) e fácil de configurar para iniciantes.
❌ Evolui mais lentamente que as concorrentes e possui menos plugins modernos.

O Paradigma Orientado a Objetos



A **Programação Orientada a Objetos (POO)** é o coração do Java. Em vez de pensar em instruções sequenciais, modelamos o mundo em **objetos** que possuem características e comportamentos.

Essa abordagem torna o código mais **organizado, reutilizável e fácil de manter** — essencial em projetos de grande escala.

- Classe
- Objeto
- Encapsulamento
- Herança
- Polimorfismo

Classe — O Molde dos Objetos

Uma classe é um **molde** que define a estrutura e o comportamento de um objeto. Ela especifica quais atributos (dados) e métodos (comportamentos) os objetos dessa classe terão.

Exemplo de código Java:

```
public class Carro {
    // Atributos
    String cor;
    String marca;
    int velocidade;

    // Método
    public void acelerar() {
        velocidade += 10;
    }
}
```

Neste exemplo, `Carro` é a classe que define como todo carro deve ser estruturado.

Objeto — A Instância da Classe

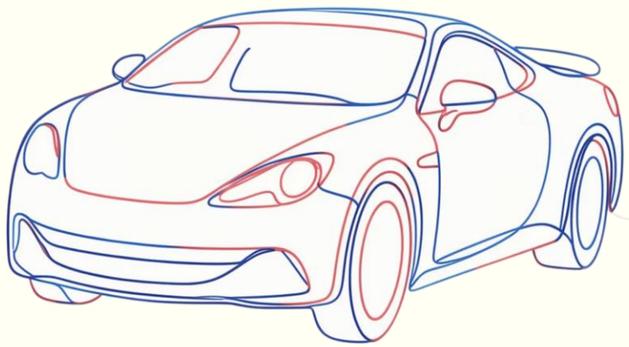
Um objeto é uma **instância** de uma classe. Enquanto a classe é o molde, o objeto é o resultado concreto criado a partir desse molde. Cada objeto possui seus próprios valores para os atributos definidos na classe.

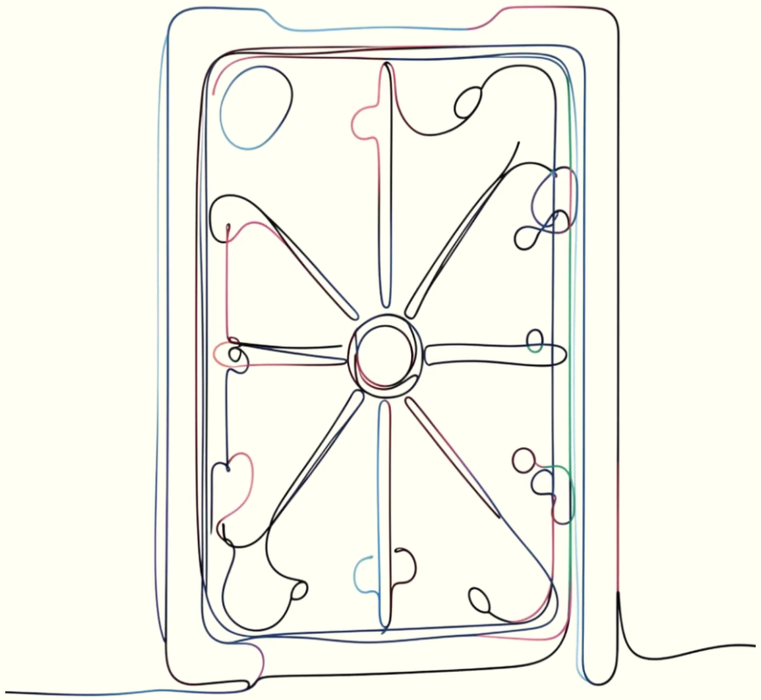
Exemplo de código Java:

```
// Criando objetos (instâncias) da classe Carro
Carro meuCarro = new Carro();
meuCarro.cor = "Vermelho";
meuCarro.marca = "Ferrari";
meuCarro.velocidade = 0;

Carro outroCarro = new Carro();
outroCarro.cor = "Azul";
outroCarro.marca = "BMW";
outroCarro.velocidade = 0;
```

Neste exemplo, `meuCarro` e `outroCarro` são dois objetos diferentes da classe `Carro`, cada um com seus próprios atributos.





Encapsulamento — Protegendo os Dados

O encapsulamento é o princípio de **ocultar os detalhes internos** de uma classe e expor apenas o que é necessário. Usamos modificadores de acesso como `private` (privado) e `public` (público) para controlar o acesso aos atributos e métodos.

Exemplo de código Java:

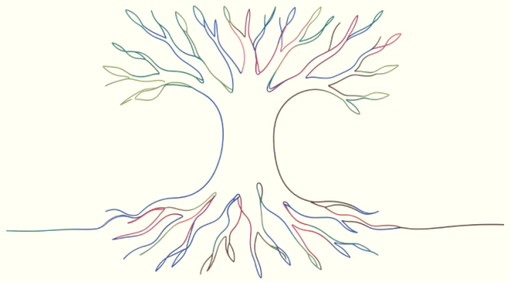
```
public class Carro {
    // Atributos privados (protegidos)
    private String cor;
    private int velocidade;

    // Métodos públicos (getters e setters)
    public String getCor() {
        return cor;
    }

    public void setCor(String novaCor) {
        cor = novaCor;
    }

    public void acelerar() {
        if (velocidade < 200) {
            velocidade += 10;
        }
    }
}
```

Neste exemplo, os atributos são privados e só podem ser acessados através dos métodos públicos, garantindo controle e segurança.



Herança — Reutilizando Código

A herança permite que uma classe **filha** herde atributos e métodos de uma classe **mãe** (superclasse). Isso promove reutilização de código e estabelece uma hierarquia entre classes.

Exemplo de código Java:

```
// Classe mãe (superclasse)
public class Veiculo {
    protected String marca;
    protected int velocidade;

    public void acelerar() {
        velocidade += 10;
    }
}

// Classe filha (subclasse)
public class Carro extends Veiculo {
    private int numPortas;

    public void abrirPorta() {
        System.out.println("Porta aberta!");
    }
}

// Usando herança
Carro meuCarro = new Carro();
meuCarro.marca = "Toyota";
meuCarro.acelerar(); // Método herdado
meuCarro.abrirPorta(); // Método próprio
```

Neste exemplo, `Carro` herda de `Veiculo` e reutiliza o método `acelerar()`, além de adicionar seu próprio método `abrirPorta()`.

Polimorfismo — Múltiplas Formas

O polimorfismo permite que o **mesmo método** tenha comportamentos diferentes dependendo do objeto que o chama. A palavra "polimorfismo" significa "muitas formas". Isso é alcançado através da sobrescrita de métodos (override).

Exemplo de código Java:

```
// Classe mãe
public class Animal {
    public void fazer_som() {
        System.out.println("Som genérico");
    }
}

// Classes filhas
public class Cachorro extends Animal {
    @Override
    public void fazer_som() {
        System.out.println("Au au!");
    }
}

public class Gato extends Animal {
    @Override
    public void fazer_som() {
        System.out.println("Miau!");
    }
}

// Usando polimorfismo
Animal animal1 = new Cachorro();
Animal animal2 = new Gato();

animal1.fazer_som(); // Saída: Au au!
animal2.fazer_som(); // Saída: Miau!
```

Neste exemplo, o método `fazer_som()` é chamado da mesma forma, mas cada classe filha implementa seu próprio comportamento.

Mercado de Trabalho Java no Brasil

Java segue sendo uma das linguagens mais valorizadas pelo mercado brasileiro, com oportunidades em grandes empresas e startups.

R$3K

Júnior

Salário médio mensal para iniciantes

R$6K

Pleno

Profissional com 2 a 5 anos de experiência

R$12K

Sênior

Desenvolvedor experiente e especialista

Onde o Java é Utilizado?

Java está em toda parte

- **Back-end Web:** construção de APIs e sistemas robustos para web
- **Mobile Android:** base histórica do desenvolvimento Android
- **Sistemas Financeiros:** Java é a linguagem soberana em grandes bancos, devido à sua segurança
- **Big Data:** plataformas como Hadoop e Apache Spark são escritas em Java
- **Sistemas Empresariais:** ERPs, CRMs e soluções corporativas de grande porte

Tecnologias e Frameworks Exigidos

O mercado vai além do Java puro — dominar o ecossistema é essencial para se destacar.

Spring Boot

Framework mais popular para criar APIs e aplicações web de forma rápida e eficiente.

Hibernate / JPA

Mapeamento objeto-relacional para persistência de dados em bancos SQL.

Docker & Cloud

Conteinerização e deploy em plataformas como AWS, Azure e GCP.

Git & Maven

Controle de versão e gerenciamento de dependências, presentes em todo projeto Java.

JUnit & Testes

Testes automatizados e garantia de qualidade.

Conclusão: Por Que Aprender Java?

🏛️ Legado sólido

Quase 30 anos de história com evolução contínua e comunidade gigante.

💼 Alta empregabilidade

Uma das linguagens mais demandadas por empresas no Brasil e no mundo.

📚 Base de conhecimento

Aprender POO em Java facilita o aprendizado de qualquer outra linguagem.